However, in the following example, the Active Record object contains just the attributes `id`, `name`, and `paytype`—only these columns can be updated when the object is saved. (Note that you have to include the `id` column if you intend to save a row fetched using `find_by_sql`.)

```
orders = Order.find_by_sql("select id, name, pay_type from orders where
id=123")
first = orders[0]
first.name = "Wilma"
first.save
```

In addition to the `save` method, Active Record lets us change the values of attributes and save a model object in a single call to `update`:

```
order = Order.find(321)
order.update(name: "Barney", email: "barney@bedrock.com")
```

The `update` method is most commonly used in controller actions where it merges data from a form into an existing database row:

```
def save_after_edit
  order = Order.find(params[:id])
  if order.update(order_params)
    redirect_to action: :index
  else
    render action: :edit
  end
end
```

We can combine the functions of reading a row and updating it using the class methods `update` and `update_all`. The `update` method takes an `id` parameter and a set of attributes. It fetches the corresponding row, updates the given attributes, saves the result to the database, and returns the model object.

```
order = Order.update(12, name: "Barney", email: "barney@bedrock.com")
```

We can pass `update` an array of IDs and an array of attribute value hashes, and it will update all the corresponding rows in the database, returning an array of model objects.